ANSI SQL Using MySQL

Project Theme: Community Event Portal

1. Database Creation and Schema Definition

```
-- Create Database
CREATE DATABASE IF NOT EXISTS CommunityEventPortal;
USE CommunityEventPortal;


-- 1. Users Table
CREATE TABLE Users (
    user_id INT PRIMARY KEY AUTO_INCREMENT,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    city VARCHAR(100) NOT NULL,
    registration_date DATE NOT NULL
);


-- 2. Events Table
CREATE TABLE Events (
    event_id INT PRIMARY KEY AUTO_INCREMENT,
    title VARCHAR(200) NOT NULL,
    description TEXT,
    city VARCHAR(100) NOT NULL,
    start_date DATETIME NOT NULL,
    end_date DATETIME NOT NULL,
    status ENUM('upcoming', 'completed', 'cancelled') NOT NULL,
    organizer_id INT,
    FOREIGN KEY (organizer_id) REFERENCES Users(user_id) ON DELETE SET NULL
);
```

```
-- 3. Sessions Table
CREATE TABLE Sessions (
    session_id INT PRIMARY KEY AUTO_INCREMENT,
    event_id INT,
    title VARCHAR(200) NOT NULL,
    speaker_name VARCHAR(100) NOT NULL,
    start_time DATETIME NOT NULL,
    end_time DATETIME NOT NULL,
    FOREIGN KEY (event_id) REFERENCES Events(event_id) ON DELETE CASCADE
);

-- 4. Registrations Table
CREATE TABLE Registrations (
    registration_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT,
    event_id INT,
    registration_date DATE NOT NULL,
    FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (event_id) REFERENCES Events(event_id) ON DELETE CASCADE
);

-- 5. Feedback Table
CREATE TABLE Feedback (
    feedback_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT,
    event_id INT,
    rating INT CHECK (rating BETWEEN 1 AND 5),
    comments TEXT,
    feedback_date DATE NOT NULL,
```

```
    FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (event_id) REFERENCES Events(event_id) ON DELETE CASCADE
);


-- 6. Resources Table
CREATE TABLE Resources (
    resource_id INT PRIMARY KEY AUTO_INCREMENT,
    event_id INT,
    resource_type ENUM('pdf', 'image', 'link') NOT NULL,
    resource_url VARCHAR(255) NOT NULL,
    uploaded_at DATETIME NOT NULL,
    FOREIGN KEY (event_id) REFERENCES Events(event_id) ON DELETE CASCADE
);
```

2. Sample Dataset Insertion

```
-- Insert Users
INSERT INTO Users (user_id, full_name, email, city, registration_date) VALUES
(1, 'Alice Johnson', 'alice@example.com', 'New York', '2024-12-01'),
(2, 'Bob Smith', 'bob@example.com', 'Los Angeles', '2024-12-05'),
(3, 'Charlie Lee', 'charlie@example.com', 'Chicago', '2024-12-10'),
(4, 'Diana King', 'diana@example.com', 'New York', '2025-01-15'),
(5, 'Ethan Hunt', 'ethan@example.com', 'Los Angeles', '2025-02-01');


-- Insert Events
INSERT INTO Events (event_id, title, description, city, start_date, end_date, status, organizer_id)
VALUES
(1, 'Tech Innovators Meetup', 'A meetup for tech enthusiasts.', 'New York', '2025-06-10 10:00:00',
'2025-06-10 16:00:00', 'upcoming', 1),
(2, 'AI & ML Conference', 'Conference on AI and ML advancements.', 'Chicago', '2025-05-15
09:00:00', '2025-05-15 17:00:00', 'completed', 3),
```

```
(3, 'Frontend Development Bootcamp', 'Hands-on training on frontend tech.', 'Los Angeles', '2025-07-01 10:00:00', '2025-07-03 16:00:00', 'upcoming', 2);


-- Insert Sessions

INSERT INTO Sessions (session_id, event_id, title, speaker_name, start_time, end_time) VALUES

(1, 1, 'Opening Keynote', 'Dr. Tech', '2025-06-10 10:00:00', '2025-06-10 11:00:00'),

(2, 1, 'Future of Web Dev', 'Alice Johnson', '2025-06-10 11:15:00', '2025-06-10 12:30:00'),

(3, 2, 'AI in Healthcare', 'Charlie Lee', '2025-05-15 09:30:00', '2025-05-15 11:00:00'),

(4, 3, 'Intro to HTML5', 'Bob Smith', '2025-07-01 10:00:00', '2025-07-01 12:00:00');


-- Insert Registrations

INSERT INTO Registrations (registration_id, user_id, event_id, registration_date) VALUES

(1, 1, 1, '2025-05-01'),

(2, 2, 1, '2025-05-02'),

(3, 3, 2, '2025-04-30'),

(4, 4, 2, '2025-04-28'),

(5, 5, 3, '2025-06-15');


-- Insert Feedback

INSERT INTO Feedback (feedback_id, user_id, event_id, rating, comments, feedback_date) VALUES

(1, 3, 2, 4, 'Great insights!', '2025-05-16'),

(2, 4, 2, 5, 'Very informative.', '2025-05-16'),

(3, 2, 1, 3, 'Could be better.', '2025-06-11');


-- Insert Resources

INSERT INTO Resources (resource_id, event_id, resource_type, resource_url, uploaded_at) VALUES

(1, 1, 'pdf', 'https://portal.com/resources/tech_meetup_agenda.pdf', '2025-05-01 10:00:00'),

(2, 2, 'image', 'https://portal.com/resources/ai_poster.jpg', '2025-04-20 09:00:00'),

(3, 3, 'link', 'https://portal.com/resources/html5_docs', '2025-06-25 15:00:00');
```

3. Exercise - SQL Queries

```
--
================================================================================
===
-- ANSI SQL USING MYSQL - COMPLETE EXERCISE SOLUTIONS
--
================================================================================
===


-- 1. User Upcoming Events
-- Show a list of all upcoming events a user is registered for in their city, sorted by date.
SELECT
    u.user_id,
    u.full_name,
    e.title AS event_title,
    e.city,
    e.start_date
FROM Registrations r
JOIN Users u ON r.user_id = u.user_id
JOIN Events e ON r.event_id = e.event_id
WHERE e.status = 'upcoming'
  AND e.city = u.city
ORDER BY e.start_date ASC;


-- 2. Top Rated Events
-- Identify events with the highest average rating, considering only those that have received at least 10
feedback submissions.
SELECT
    e.event_id,
    e.title,
    AVG(f.rating) AS avg_rating,
    COUNT(f.feedback_id) AS total_feedback
```

```
FROM Events e
JOIN Feedback f ON e.event_id = f.event_id
GROUP BY e.event_id, e.title
HAVING COUNT(f.feedback_id) >= 10
ORDER BY avg_rating DESC;

-- 3. Inactive Users
-- Retrieve users who have not registered for any events in the last 90 days.
SELECT
    u.user_id,
    u.full_name,
    u.email
FROM Users u
WHERE u.user_id NOT IN (
    SELECT DISTINCT user_id
    FROM Registrations
    WHERE registration_date >= CURDATE() - INTERVAL 90 DAY
);

-- 4. Peak Session Hours
-- Count how many sessions are scheduled between 10 AM to 12 PM for each event.
SELECT
    e.event_id,
    e.title,
    COUNT(s.session_id) AS peak_sessions_count
FROM Events e
LEFT JOIN Sessions s ON e.event_id = s.event_id
    AND TIME(s.start_time) >= '10:00:00'
    AND TIME(s.end_time) <= '12:00:00'
GROUP BY e.event_id, e.title;
```

```
-- 5. Most Active Cities
-- List the top 5 cities with the highest number of distinct user registrations.
SELECT
    u.city,
    COUNT(DISTINCT r.user_id) AS distinct_registered_users
FROM Registrations r
JOIN Users u ON r.user_id = u.user_id
GROUP BY u.city
ORDER BY distinct_registered_users DESC
LIMIT 5;

-- 6. Event Resource Summary
-- Generate a report showing the number of resources (PDFs, images, links) uploaded for each event.
SELECT
    e.event_id,
    e.title,
    SUM(CASE WHEN res.resource_type = 'pdf' THEN 1 ELSE 0 END) AS pdf_count,
    SUM(CASE WHEN res.resource_type = 'image' THEN 1 ELSE 0 END) AS image_count,
    SUM(CASE WHEN res.resource_type = 'link' THEN 1 ELSE 0 END) AS link_count,
    COUNT(res.resource_id) AS total_resources
FROM Events e
LEFT JOIN Resources res ON e.event_id = res.event_id
GROUP BY e.event_id, e.title;

-- 7. Low Feedback Alerts
-- List all users who gave feedback with a rating less than 3, along with their comments and associated event names.
SELECT
    u.full_name,
```

```
    e.title AS event_name,
    f.rating,
    f.comments
FROM Feedback f
JOIN Users u ON f.user_id = u.user_id
JOIN Events e ON f.event_id = e.event_id
WHERE f.rating < 3;


-- 8. Sessions per Upcoming Event
-- Display all upcoming events with the count of sessions scheduled for them.
SELECT
    e.event_id,
    e.title,
    COUNT(s.session_id) AS session_count
FROM Events e
LEFT JOIN Sessions s ON e.event_id = s.event_id
WHERE e.status = 'upcoming'
GROUP BY e.event_id, e.title;


-- 9. Organizer Event Summary
-- For each event organizer, show the number of events created and their current status (upcoming, completed, cancelled).
SELECT
    u.user_id AS organizer_id,
    u.full_name AS organizer_name,
    e.status,
    COUNT(e.event_id) AS total_events
FROM Users u
JOIN Events e ON u.user_id = e.organizer_id
GROUP BY u.user_id, u.full_name, e.status;
```

```sql
-- 10. Feedback Gap
-- Identify events that had registrations but received no feedback at all.
SELECT DISTINCT
    e.event_id,
    e.title
FROM Events e
JOIN Registrations r ON e.event_id = r.event_id
LEFT JOIN Feedback f ON e.event_id = f.event_id
WHERE f.feedback_id IS NULL;

-- 11. Daily New User Count
-- Find the number of users who registered each day in the last 7 days.
SELECT
    registration_date,
    COUNT(user_id) AS new_users_count
FROM Users
WHERE registration_date >= CURDATE() - INTERVAL 7 DAY
GROUP BY registration_date
ORDER BY registration_date DESC;

-- 12. Event with Maximum Sessions
-- List the event(s) with the highest number of sessions.
SELECT
    e.event_id,
    e.title,
    COUNT(s.session_id) AS total_sessions
FROM Events e
JOIN Sessions s ON e.event_id = s.event_id
GROUP BY e.event_id, e.title
```

```
HAVING COUNT(s.session_id) = (
    SELECT MAX(session_count)
    FROM (
        SELECT COUNT(session_id) AS session_count
        FROM Sessions
        GROUP BY event_id
    ) AS counts
);

-- 13. Average Rating per City
-- Calculate the average feedback rating of events conducted in each city.
SELECT
    e.city,
    ROUND(AVG(f.rating), 2) AS avg_city_rating
FROM Events e
JOIN Feedback f ON e.event_id = f.event_id
GROUP BY e.city;

-- 14. Most Registered Events
-- List top 3 events based on the total number of user registrations.
SELECT
    e.event_id,
    e.title,
    COUNT(r.registration_id) AS total_registrations
FROM Events e
JOIN Registrations r ON e.event_id = r.event_id
GROUP BY e.event_id, e.title
ORDER BY total_registrations DESC
LIMIT 3;
```

```sql
-- 15. Event Session Time Conflict
-- Identify overlapping sessions within the same event (i.e., session start and end times that conflict).
SELECT
    s1.event_id,
    s1.session_id AS session1_id,
    s1.title AS session1_title,
    s2.session_id AS session2_id,
    s2.title AS session2_title
FROM Sessions s1
JOIN Sessions s2 ON s1.event_id = s2.event_id
                AND s1.session_id < s2.session_id
WHERE s1.start_time < s2.end_time
  AND s1.end_time > s2.start_time;

-- 16. Unregistered Active Users
-- Find users who created an account in the last 30 days but haven't registered for any events.
SELECT
    u.user_id,
    u.full_name,
    u.registration_date
FROM Users u
LEFT JOIN Registrations r ON u.user_id = r.user_id
WHERE u.registration_date >= CURDATE() - INTERVAL 30 DAY
  AND r.registration_id IS NULL;

-- 17. Multi-Session Speakers
-- Identify speakers who are handling more than one session across all events.
SELECT
    speaker_name,
    COUNT(session_id) AS total_sessions
```

```
FROM Sessions
GROUP BY speaker_name
HAVING COUNT(session_id) > 1;


-- 18. Resource Availability Check
-- List all events that do not have any resources uploaded.
SELECT
    e.event_id,
    e.title
FROM Events e
LEFT JOIN Resources r ON e.event_id = r.event_id
WHERE r.resource_id IS NULL;


-- 19. Completed Events with Feedback Summary
-- For completed events, show total registrations and average feedback rating.
SELECT
    e.event_id,
    e.title,
    COUNT(DISTINCT r.registration_id) AS total_registrations,
    ROUND(AVG(f.rating), 2) AS avg_rating
FROM Events e
LEFT JOIN Registrations r ON e.event_id = r.event_id
LEFT JOIN Feedback f ON e.event_id = f.event_id
WHERE e.status = 'completed'
GROUP BY e.event_id, e.title;


-- 20. User Engagement Index
-- For each user, calculate how many events they attended and how many feedbacks they submitted.
SELECT
    u.user_id,
```

```
    u.full_name,
    COUNT(DISTINCT r.event_id) AS events_registered,
    COUNT(DISTINCT f.feedback_id) AS feedbacks_submitted
FROM Users u
LEFT JOIN Registrations r ON u.user_id = r.user_id
LEFT JOIN Feedback f ON u.user_id = f.user_id
GROUP BY u.user_id, u.full_name;

-- 21. Top Feedback Providers
-- List top 5 users who have submitted the most feedback entries.
SELECT
    u.user_id,
    u.full_name,
    COUNT(f.feedback_id) AS feedback_count
FROM Users u
JOIN Feedback f ON u.user_id = f.user_id
GROUP BY u.user_id, u.full_name
ORDER BY feedback_count DESC
LIMIT 5;

-- 22. Duplicate Registrations Check
-- Detect if a user has been registered more than once for the same event.
SELECT
    user_id,
    event_id,
    COUNT(registration_id) AS registration_count
FROM Registrations
GROUP BY user_id, event_id
HAVING COUNT(registration_id) > 1;
```

```sql
-- 23. Registration Trends
-- Show a month-wise registration count trend over the past 12 months.
SELECT
    DATE_FORMAT(registration_date, '%Y-%m') AS registration_month,
    COUNT(registration_id) AS total_registrations
FROM Registrations
WHERE registration_date >= CURDATE() - INTERVAL 12 MONTH
GROUP BY DATE_FORMAT(registration_date, '%Y-%m')
ORDER BY registration_month ASC;


-- 24. Average Session Duration per Event
-- Compute the average duration (in minutes) of sessions in each event.
SELECT
    e.event_id,
    e.title,
    ROUND(AVG(TIMESTAMPDIFF(MINUTE, s.start_time, s.end_time)), 2) AS
avg_duration_minutes
FROM Events e
JOIN Sessions s ON e.event_id = s.event_id
GROUP BY e.event_id, e.title;


-- 25. Events Without Sessions
-- List all events that currently have no sessions scheduled under them.
SELECT
    e.event_id,
    e.title
FROM Events e
LEFT JOIN Sessions s ON e.event_id = s.event_id
WHERE s.session_id IS NULL;
```